RAG Architecture

1. RAG in one sentence

Retrieval-augmented generation: retrieve relevant chunks from your documents and give them to the model to answer from. It changes what the model *knows at answer time*, so updating knowledge means updating the index, not the model.

2. Naive RAG breaks four ways

1 **Vague queries** retrieve bad chunks (conversational questions sit far from document language)

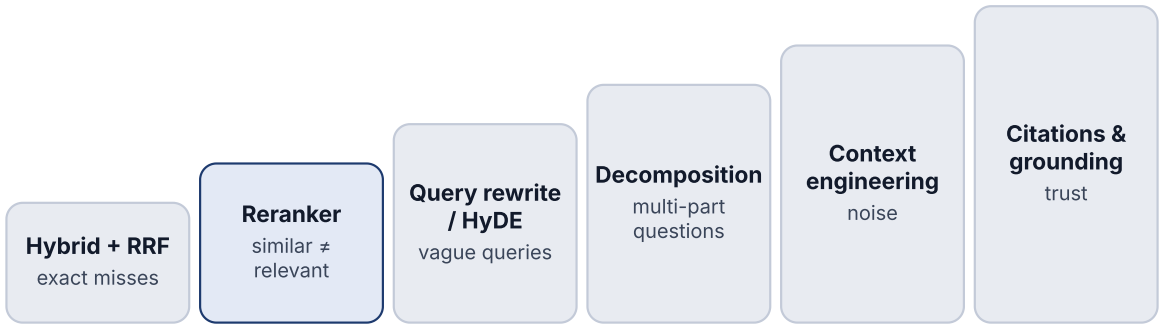
2 **Similar ≠ relevant**: the most similar chunks often don't contain the answer (unreranked top-K carries 20 to 40% noise)

3 **Exact-match misses**: SKUs, table names, error codes

4 **Noisy or contradictory context**: five chunks in, two relevant, and the model confidently synthesises from the noise

3. The production ladder (build in this order)

✎ DIAGRAM



Each rung fixes a named failure. Measure after each one; the reranker is usually the biggest single gain.

Present RAG as targeted fixes for named failures, not as fashion.

- **Reranker (cross-encoder)**: reads the question and each candidate together to score true relevance. Retrieve ~50 cheaply, rerank to the best 5. Commonly +10 to 20% quality
- **HyDE**: a cheap model drafts a plausible answer and you search with *that* embedding, because answers sit near answers
- **Context engineering**: best chunk first, filter hard (three good chunks beat five mediocre), dedupe, number the chunks so the model can cite them

- **Grounding & citations**: answer only from the chunks, cite each claim, say 'I don't know' otherwise

4. Why citations are worth paying for

- **Trust**: users can verify · **Debuggability**: trace a wrong answer to the chunk that caused it · **Quality**: enables automated faithfulness checks · **Compliance**: an audit trail

5. Evaluate in three places

- **Retrieval** → recall@K on a hand-labelled question set (the most important number)
- **Context** → right chunks, right order
- **Generation** → faithfulness (every claim traceable to a chunk), scored by a calibrated LLM judge

6. Debugging a confidently wrong answer

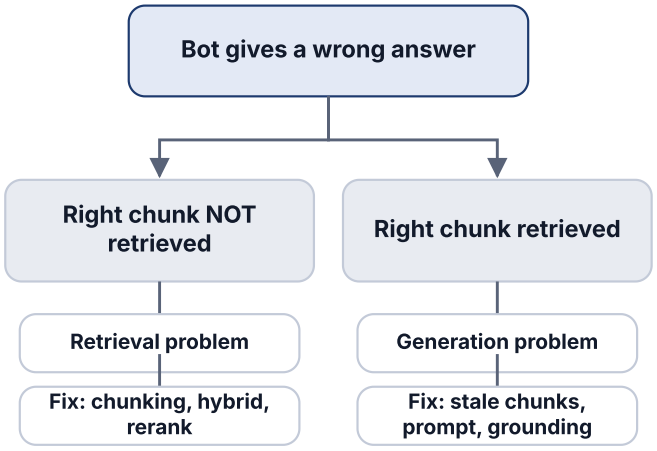
1 **Log what was retrieved** for that exact question

2 Right chunk missing → **retrieval** problem: parsing, chunking, embedding model, top-K, hybrid search, reranker

3 Right chunk present, answer still wrong → **generation** problem: conflicting or stale chunks, weak grounding instructions, model ignoring context

4 Turn the failure into a permanent test case in the golden set

✎ DIAGRAM



Split retrieval failures from generation failures before touching anything.

7. RAG vs fine-tuning vs long context

- **RAG = knowledge** (facts that change, need citations, per-user permissions)

- **Fine-tuning = behaviour** (format, tone, a narrow task, or distilling a big model into a small cheap one)
- **Long context + caching** = a small (under ~500k tokens), stable corpus with no pipeline to build
- **Prompting first** for both. They combine: a fine-tuned small model on top of RAG is common
- Long-context tax: a 200k-token call costs and takes roughly 10× a 20k call, even when it fits
- True cost of fine-tuning: the labelled data, the eval set to prove it helped, retraining on every base-model change, and lock-in

> ⚠ Fine-tuning to 'teach the model our documents' fails: facts get blurred, can't be cited, go stale, and can't be removed per user.

✎ DIAGRAM



RAG for knowledge, fine-tuning for behaviour, prompting first for both.

8. How models are trained and tuned

1 **Pre-training**: predict the next token over trillions of tokens → knowledge, not manners

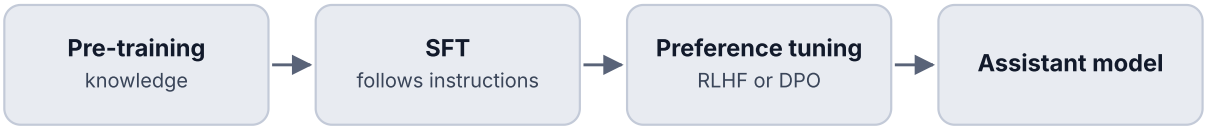
2 **SFT** (supervised fine-tuning): example prompts + ideal answers → follows instructions

3 **Preference tuning**: **RLHF** (train a reward model on human rankings, then optimise against it) or **DPO** (learn directly from better-vs-worse pairs, simpler)

- **LoRA**: train small add-on matrices instead of all weights · **QLoRA**: LoRA on a quantised base, fits big models on modest GPUs · **Distillation**: a big model teaches a small one a narrow task
- **Quantisation**: weights in fewer bits (16 → 8 → 4), less memory, faster, small quality loss

- Risk: **catastrophic forgetting** (narrow tuning degrades general skills)

✎ DIAGRAM



How a raw next-token predictor becomes a helpful assistant.

SAY IT

"RAG for knowledge, fine-tuning for behaviour, prompting first for both. When the bot is wrong, I first check what was retrieved, because that tells me whether it's a retrieval or a generation problem."